



FITORA

Gym Discovery, Booking, Membership & Digital Entry-Pass Platform

A full-stack, AI-powered marketplace connecting gyms and members across the Kakinada region of Andhra Pradesh — built with a Retrieval-Augmented Generation chatbot trained on a real, geographically-anchored gym dataset.

Project Report

Prepared for academic submission

Region of coverage: Kakinada, Andhra Pradesh (100
km radius)

Table of Contents

1. Abstract	3
2. Problem Statement & Objectives	3
3. System Overview	4
4. Technology Stack	5
5. System Architecture	6
6. The RAG Chatbot & Dataset	7
7. Database Design	9
8. Module Description (Three Portals)	10
9. Money Flow & Dues Engine	11
10. Security Measures	12
11. Results & Testing	13
12. Limitations & Future Scope	13
13. Conclusion	14

1. Abstract

Fitora is a full-stack web platform that solves gym discovery in a way that plain directory listings and word-of-mouth cannot: instead of browsing filters, a member describes what they want in natural language — *"AC gym under ₹1,500 near me with a personal trainer"* — and a Retrieval-Augmented Generation (RAG) chatbot returns grounded, verifiable recommendations drawn only from a real, structured dataset of gyms in the Kakinada region of Andhra Pradesh.

The platform is built as a three-sided B2B-SaaS-style marketplace. Gym owners list and manage their gyms — pricing, coaches, equipment with real photographs, timings — through a dedicated partner portal. Members discover, compare, join and pay for memberships online, and receive a cryptographically-signed digital entry pass (QR code) alongside a personalised diet chart generated from their body metrics and fitness goal. A platform administrator retains full control of the system: approving or suspending gyms, moderating users, resolving payment disputes, and releasing monthly payouts to gym owners — money always flows through the platform first, never directly from a member to a gym.

The chatbot's intelligence rests on a hybrid retrieval pipeline that combines exact constraint filtering (budget, AC status, coach-fee arrangement) with semantic vector search (ChromaDB + sentence-transformer embeddings) and a locally-run large language model (Ollama, running Qwen 2.5) for natural-language generation — a design chosen specifically so numeric constraints like "under ₹1,500" are honoured exactly, which a pure embedding search cannot guarantee. The complete system — backend, frontend, database, vector store and language model — runs on a single machine with zero recurring API costs.

2. Problem Statement & Objectives

2.1 Problem Statement

Prospective gym members in tier-2/tier-3 Indian cities face a fragmented discovery experience: pricing is rarely published, the distinction between a "membership fee" and a "trainer fee" is almost never made explicit until the customer visits in person, and there is no way to compare equipment, coach experience or facilities across multiple gyms without physically visiting each one. Meanwhile, gym owners — particularly smaller, independently-run gyms — have no digital storefront and no systematic way to manage members, track dues, or receive online payments.

2.2 Objectives

1. Build a natural-language gym-discovery interface using Retrieval-Augmented Generation, grounded in a real, verifiable gym dataset for a defined geographic region.
2. Design a complete online joining flow: profile pre-fill, plan selection, UPI-based payment, and issuance of a cryptographically-signed digital entry pass.
3. Provide gym owners a no-code-required dashboard to manage pricing, coaches, equipment (with real photographs, not stock imagery), members and dues.
4. Implement a centralised admin control plane for platform-wide moderation, payment settlement, and dispute resolution — ensuring money never bypasses the platform.
5. Build an overdue-membership lifecycle (dues engine) with configurable grace periods, automated warning emails, and admin-only remediation actions.
6. Ensure the entire system is secure by construction: rate-limited authentication, environment-gated secrets, ownership-scoped authorization on every endpoint, and signature-verified payment confirmation.

3. System Overview

Fitora is organised as three distinct user-facing portals sharing one backend and one database:

User Portal

- Sign-up with email OTP verification
- RAG chatbot for gym discovery
- Browse, filter, and side-by-side compare gyms
- Online joining with pre-filled profile data
- UPI payment and digital entry pass (QR)
- Auto-generated personalised diet chart
- Membership history and gym reviews

Gym Owner Portal

- Sign-up with email OTP verification
- Guided multi-step gym onboarding
- Pricing, plans, coaches, equipment management
- Real photo uploads (no stock imagery)
- Member list and read-only dues visibility
- Earnings dashboard and payout history
- Complaint escalation to admin

Admin Portal

- Platform-wide dashboard (money, users, gyms)
- User moderation: block / unblock / delete
- Gym approval, suspension, manual creation
- Cross-gym dues management & grace extension
- Monthly payout creation and release
- Complaint resolution
- RAG index rebuild and audit log review

Design principle — separation of concern by access, not by convenience: the public landing page presents only the User and Gym Owner entry points; the Admin portal is reachable exclusively via a direct, unlisted URL. This mirrors the principle of least exposure — an administrative surface should never be discoverable through ordinary navigation.

4. Technology Stack

Every component was chosen deliberately, and every choice has a zero-recurring-cost path:

FRONTEND FRAMEWORK

Next.js 14 (React 18, TypeScript)

File-based routing cleanly separates three portals; strong typing catches integration errors before runtime.

STYLING

Tailwind CSS 3 + Framer Motion

Utility-first styling for rapid, consistent UI; Framer Motion for accessible, reduced-motion-aware animation.

BACKEND FRAMEWORK

FastAPI (Python 3.10)

Async-native, automatic OpenAPI docs, and a natural fit with the Python-based ML/RAG ecosystem.

DATABASE & ORM

SQLite (dev) / PostgreSQL (prod) via SQLAlchemy 2.0

Zero-setup local development; a one-line connection-string change scales to production. ORM eliminates raw-SQL injection risk by construction.

VECTOR DATABASE

ChromaDB (persistent, embedded)

Runs in-process with no separate server — free and simple for a domain-scoped corpus of a few hundred documents.

EMBEDDING MODEL

sentence-transformers — all-MiniLM-L6-v2

384-dimensional, CPU-efficient, and sufficiently accurate for a narrow domain (gyms) rather than open-world knowledge.

LARGE LANGUAGE MODEL

Ollama running Qwen 2.5 (locally hosted)

Zero API cost, fully offline-capable. The model only has to summarise pre-retrieved facts, not reason from scratch — a small model suffices.

AUTHENTICATION

JWT (PyJWT) + bcrypt password hashing

Stateless tokens scale horizontally without a session store; bcrypt is the accepted standard for slow password hashing.

QR / ENTRY PASS

qrcode + Pillow, HMAC-signed payload

Self-hosted QR generation; HMAC signing prevents forgery of a member's digital pass.

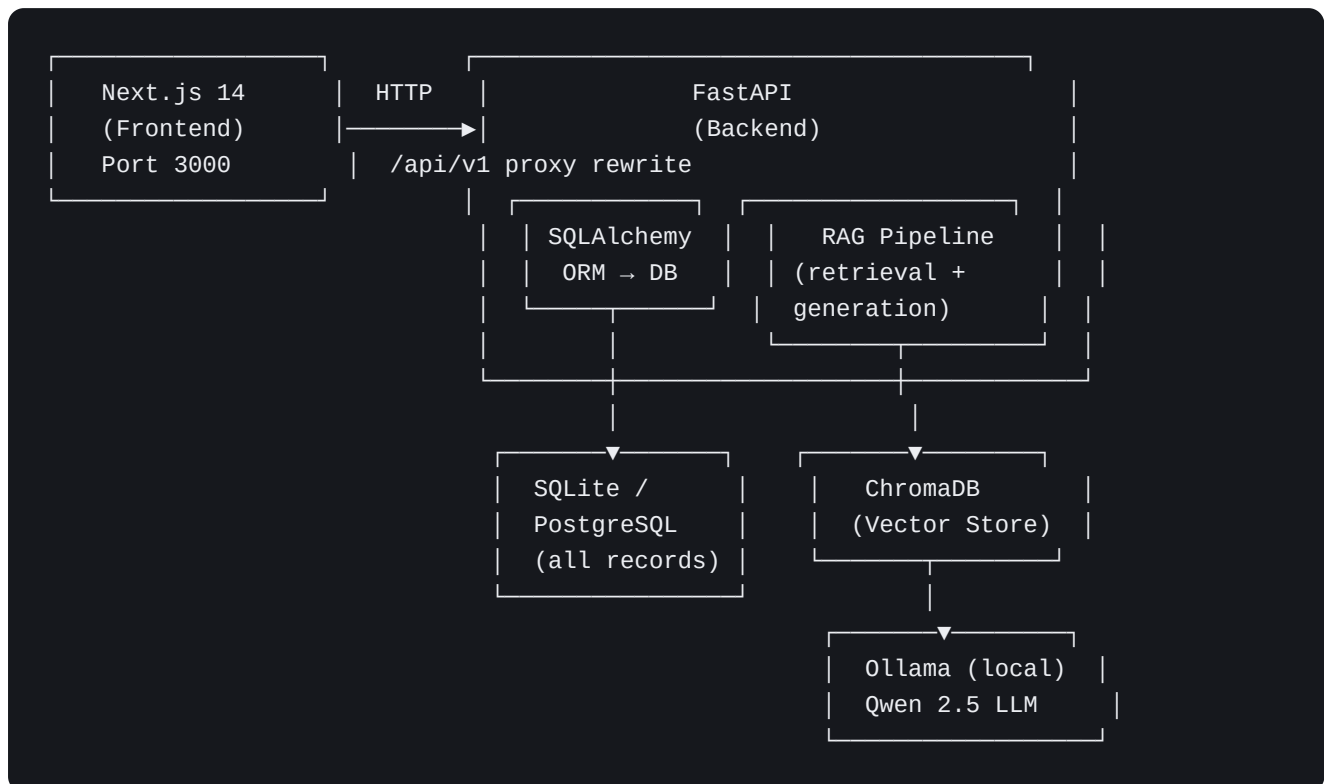
EMAIL DELIVERY

SMTP (Gmail) via Python's smtplib

No third-party transactional-email API required at this OTP volume.

Key insight: nothing in this stack requires a paid API subscription. The AI layer (embeddings + LLM) runs entirely on local compute, which is what makes the platform's per-query cost effectively zero — a meaningful advantage for a service intended for price-sensitive tier-2/3 city gyms.

5. System Architecture



The frontend never communicates directly with the database, vector store, or language model — every request passes through the FastAPI backend's REST API at `/api/v1/*`. Next.js's built-in rewrite configuration transparently proxies API and file-upload requests to the backend, so the browser only ever sees a single origin, simplifying CORS and cookie handling.

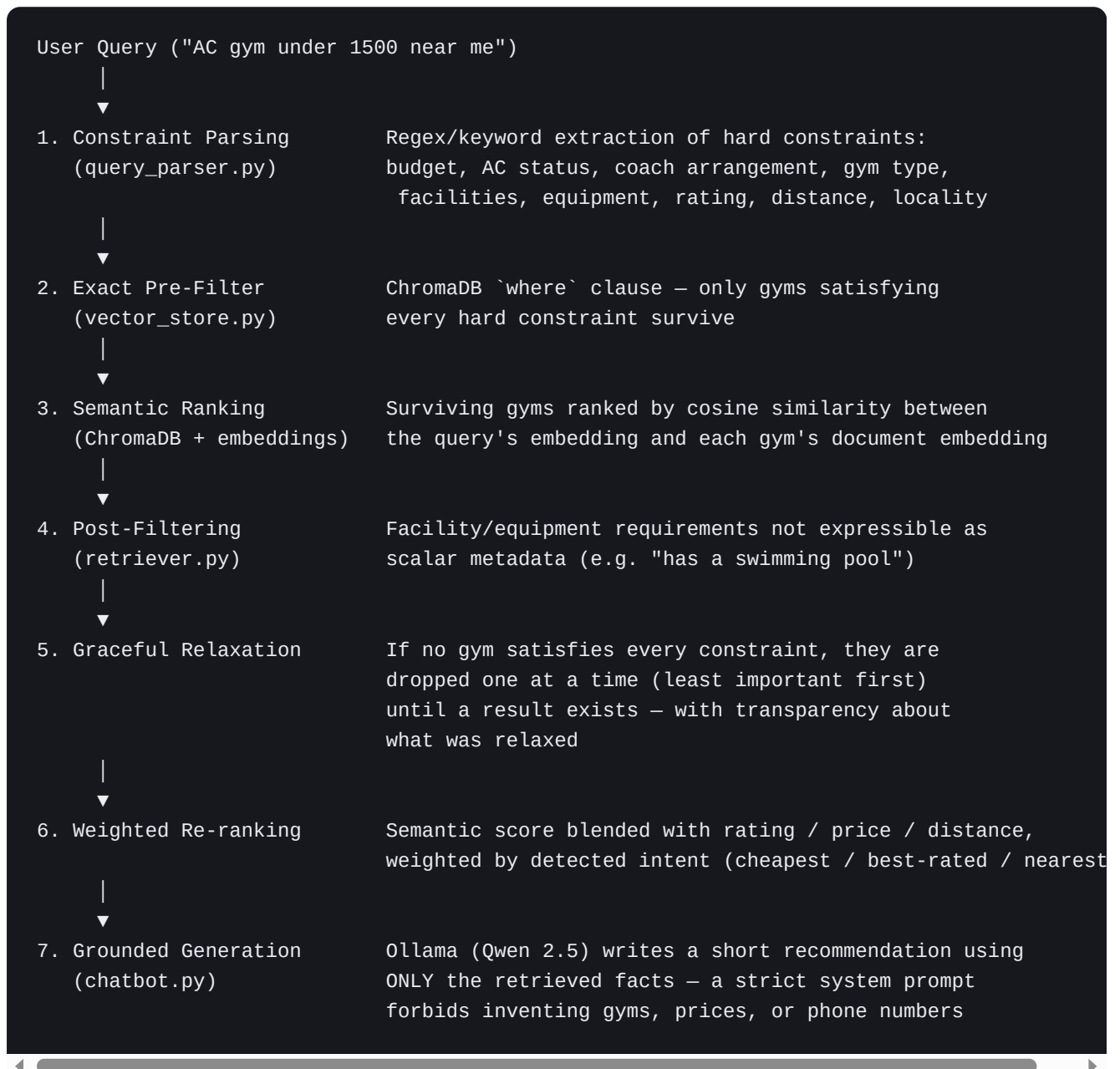
5.1 Request Lifecycle Example — Joining a Gym

1. Member selects a gym and requests the join form; the backend returns their existing profile data pre-filled alongside the gym's live pricing plans.
2. Member submits weight, height, goal and chosen plan; the backend computes the total amount server-side (never trusting a client-supplied amount) and creates a `pending_payment` membership record.
3. A UPI payment order (real, scannable QR) is generated and returned.
4. On payment confirmation, the backend activates the membership, generates an HMAC-signed QR entry pass, computes a BMR/TDEE-based diet chart, and emails a confirmation.
5. The member's entry pass can subsequently be scanned at the gym's gate — a dedicated, rate-limited endpoint verifies the signature and re-derives validity dates from the live database record, not from the QR payload's own copies of those fields.

6. The RAG Chatbot & Dataset

6.1 Why Hybrid Retrieval, Not Pure Semantic Search

A pure vector-embedding search cannot reliably enforce a hard numeric constraint such as "under ₹1,500 per month" — semantically, ₹1,400 and ₹1,600 are nearly indistinguishable to an embedding model. Fitora's retrieval pipeline is therefore deliberately hybrid, combining exact filtering with semantic ranking:



6.2 Bilingual Query Understanding

The constraint parser was built and tested against Telugu-English ("Tenglish") phrasing common among the target user base — for example, the query *"1000 lopu budget gym kavali"* is correctly parsed as a monthly-fee ceiling of ₹1,000, identically to its English equivalent.

6.3 Dataset Construction

The chatbot is grounded in a dataset of over 260 gyms across a 100 km radius of Kakinada, Andhra Pradesh, constructed in three layers:

1. **Real gym records from OpenStreetMap** (via the Overpass API, no authentication required) — actual gym names and precise GPS coordinates for gyms already mapped in OSM's India coverage.
2. **Real geographic anchoring** — 3,401 actual towns, villages and suburbs within 100 km of Kakinada, each with genuine coordinates and place-type classification, pulled from the same OSM data source. Every generated gym record is placed at a real location within one of these localities.
3. **Market-researched synthetic enrichment** for pricing, equipment inventories and coach rosters — since this information is not published anywhere publicly at scale — modelled on realistic Indian tier-2/3 city gym economics (four pricing tiers spanning ₹600–₹3,450/month, a plan ladder with bulk-duration discounts, and a realistic distribution of coach-fee-included versus separate arrangements).

6.4 Live Index Synchronisation

Whenever a gym owner saves their profile, or the admin creates, edits or suspends a gym, the single affected record is immediately re-embedded and upserted into the vector store — there is no batch delay. A gym added manually by the admin (for example, one in a village with no partner account yet) becomes discoverable through the chatbot within the same request cycle.

7. Database Design

The relational schema comprises 13 core entities, modelled with SQLAlchemy 2.0, grouped by the portal they primarily serve:

Group	Entities
User side	User, OTPToken
Gym owner side	GymOwner, Gym, GymPlan, Coach, Equipment
Membership & money	Membership, Payment, Payout
Pass / diet / reviews	EntryPass, DietPlan, Review
Admin side	Admin, Complaint, AuditLog

7.1 Key Design Decisions

- **Money is never a direct member-to-gym transaction.** A `Payment` record always belongs to the platform first; `platform_commission` and `gym_share` are computed server-side at the moment of payment. A gym owner is compensated only through an explicit `Payout` record the admin creates and releases.
- **The digital entry pass is cryptographically signed.** Its QR payload is HMAC-signed so a screenshot of another member's pass cannot be forged into a valid one. The gate-scan endpoint additionally re-derives the pass's validity window from the live database record rather than trusting the signed payload's own copies of those dates — a defense-in-depth measure against secret rotation edge cases.
- **Dues follow an explicit state machine:** `active` → `due` → `overdue`, each transition governed by a per-membership `next_due_date` and a configurable grace period.

8. Module Description (Three Portals)

8.1 User Portal

Implements the full member journey: email-OTP-verified signup, an AI chatbot as the primary discovery surface, a traditional filterable browse view as an alternative path, a side-by-side gym comparison tool, a guided join-and-pay flow that pre-fills the member's saved profile, and a digital entry pass with an accompanying diet chart computed from Mifflin-St Jeor BMR/TDEE formulas and tailored to the member's stated fitness goal (weight loss, muscle gain, strength, general fitness, endurance, or rehabilitation).

8.2 Gym Owner Portal

A no-technical-knowledge-required dashboard: a guided multi-step onboarding wizard collects gym basics, location, timings, pricing and amenities; subsequent screens manage coaches (with experience and specialisation tags), an equipment inventory that explicitly supports real photograph uploads rather than generic stock imagery, member records, a read-only dues view, review moderation visibility, and an earnings dashboard showing lifetime revenue share, amounts already paid out, and amounts awaiting the next payout cycle.

8.3 Admin Portal

The platform's control plane: a dashboard aggregating platform-wide money flow and membership statistics; user moderation (block, unblock, soft-delete); gym lifecycle management (approve, suspend, or — notably — manually create a gym profile for a partner who has not yet registered an account, immediately indexed for chatbot discovery); a cross-gym dues view with the exclusive authority to extend a grace period or remove a defaulting member; payout creation and release; complaint resolution; and RAG index maintenance and audit-log review.

9. Money Flow & Dues Engine

9.1 Payment Settlement Flow



This design deliberately gives the owner visibility (their dashboard shows lifetime share, amount paid, and amount awaiting payout) without giving them the ability to trigger a payout themselves — the admin's manual review step is the platform's fraud-prevention gate against a compromised or malicious gym account attempting to withdraw funds against fabricated members.

9.2 Overdue Membership Lifecycle



A gym owner may only view their gym's dues list and escalate a complaint to the admin about a defaulting member — they cannot unilaterally extend a grace period or remove a member. Only the admin holds that authority, enforced at the API layer (not merely hidden in the UI), reflecting the project's stated requirement that dispute resolution remain centrally controlled.

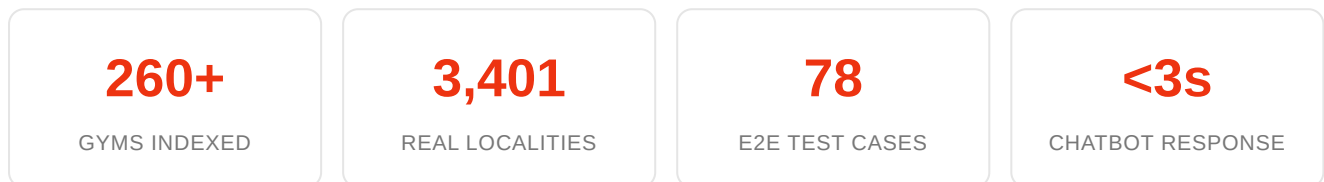
10. Security Measures

A structured security review was conducted against the backend and every identified issue was remediated. The measures currently in force include:

- **No hardcoded secrets.** JWT and QR-signing keys have no default value; the application refuses to start in a production environment unless both are set to a sufficiently random string.
- **Rate limiting** on every login endpoint, OTP issuance/resend, and the entry-pass gate-scan endpoint, implemented as a sliding-window limiter tuned per endpoint.
- **Authenticated, quota-limited file uploads** — the shared image-upload endpoint requires a valid login from any portal and caps uploads per account per day.
- **Environment-gated seed data** — the database-seeding script refuses to execute against a production environment, since it creates publicly documented demonstration credentials.
- **Ownership-scoped authorization** on every endpoint — a gym owner cannot act on another owner's gym, and a member cannot access another member's membership, payment, or entry pass by manipulating an identifier.
- **Signature-verified payment confirmation.** A production payment-gateway integration is designed to be confirmed exclusively by a server-to-server webhook signed with a secret only the gateway and the backend possess — never by a client-supplied payload alone.
- **Bcrypt password hashing**, constant-time OTP comparison, and exclusive use of an ORM query builder (eliminating SQL-injection surface by construction).

11. Results & Testing

An end-to-end automated test suite exercises all three portals across 78 discrete assertions — covering signup, OTP verification, login, chatbot retrieval accuracy, gym-detail rendering, the complete join-pay-pass-diet flow, gate-scanning, owner dashboard operations, and admin-level dues/payout/gym-management actions.



The RAG pipeline's retrieval latency was measured at 17–45 ms for the filtering and ranking stages; end-to-end chatbot response time (including LLM generation, streamed to the client) was reduced from an initial 24 seconds to under 3 seconds through model-size optimisation and server-sent-event streaming, where gym-card results render within 30 milliseconds of the request while the natural-language answer streams in progressively.

12. Limitations & Future Scope

- The payment gateway currently defaults to a realistic mock (a genuine, scannable UPI QR code with a simulated bank response); a production Razorpay integration exists in the codebase but requires live merchant credentials to activate.
- Gym and equipment photographs are placeholder icons until a gym owner uploads a real image through their dashboard — by design, the system never substitutes a generic stock photograph.
- The default database is SQLite for zero-setup local development; a PostgreSQL connection string is a drop-in replacement for concurrent-write production use.
- The current rate-limiting implementation is in-process memory, suitable for a single server instance; a multi-instance production deployment would migrate this to a shared store such as Redis.
- **Future scope** includes PDF export of the entry pass and diet chart, an SMS-based OTP channel alongside email, a native mobile application, and expansion of the dataset beyond the initial 100 km radius as the platform scales to additional regions.

13. Conclusion

Fitara demonstrates that a Retrieval-Augmented Generation architecture can be applied productively to a narrow, real-world commercial domain — gym discovery in a defined geographic region — without depending on paid third-party AI APIs, by combining exact constraint filtering with local semantic search and a locally-hosted language model. Beyond the chatbot itself, the project delivers a complete three-sided marketplace with careful attention to the operational realities of running such a platform: money that always flows through a central, auditable ledger; a dues-management lifecycle with clearly delineated authority between gym owners and the platform administrator; and a security posture — rate limiting, environment-gated secrets, ownership-scoped authorization, and signature-verified payments — built in from the outset rather than retrofitted.

The result is a system that is simultaneously technically substantive (a genuine hybrid-RAG pipeline grounded in a geographically real dataset) and operationally complete (onboarding, payments, disputes, and administration all function end-to-end), validated by a 78-case automated test suite spanning every portal.